```
import numpy as np
```

Purpose: Imports the **NumPy** (Numerical Python) library into the current environment. NumPy is the foundational package for scientific computing in Python, providing support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays. The `as np` part creates a standard alias, allowing you to call NumPy functions using `np.function()` instead of the full name.

Input: None

Output: The `np` namespace is initialized, making all NumPy modules, classes, and methods available for use in the notebook.

```
import pandas as pd
```

Purpose: Imports the **pandas** library, the primary tool for data manipulation and analysis in Python. It provides high-performance, easy-to-use data structures like DataFrames (tabular data) and Series (1D arrays). The alias `pd` is the industry standard, allowing for concise access to the library's functions.

Input: None

Output: The `pd` namespace is initialized, enabling the creation, loading, and manipulation of structured data within the notebook.

```
import matplotlib.pyplot as plt
```

Purpose: Imports the **pyplot** module from the **Matplotlib** library. This module provides a MATLAB-like interface for creating a wide range of static, animated, and interactive visualizations in Python, such as line plots, scatter plots, and histograms. The alias `plt` is the standard shorthand used to access plotting commands efficiently.

Input: None (accesses the Matplotlib library installed in your Python environment).

Output: The `plt` namespace is initialized, allowing you to generate and customize figures and axes to visualize your data.

```
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
```

Purpose: Imports specific functions and classes from the **Scikit-Learn** model_selection module to handle how data is divided and validated during the machine learning process.

`train_test_split`: Used to partition a single dataset into two sets (training and testing) to evaluate model performance on unseen data.

`StratifiedKFold`: A cross-validation object that ensures each "fold" (subset of data) has the same percentage of samples of each target class as the complete set, preventing biased results in imbalanced data.

`cross_validate`: A tool that automates the process of training and evaluating a model multiple times across different data splits to provide a more robust performance estimate.

Input: None

Output: Loads these three specific utilities into the local namespace, making them ready to be called directly in your code without a prefix.

```
from sklearn.impute import SimpleImputer
```

Purpose: Imports the `SimpleImputer` class, which is a fundamental tool for data cleaning. It is used to handle missing values (NaNs) in a dataset by replacing them with a specific placeholder, such as the mean, median, most frequent value (mode), or a constant. This is crucial because most machine learning algorithms cannot handle datasets with missing entries.

Input: None

Output: Loads the `SimpleImputer` class into the local namespace, allowing you to instantiate an imputer object to "fill in the blanks" in your data.

```
from sklearn.preprocessing import StandardScaler
```

Purpose: Imports the `StandardScaler` class, which is used for feature scaling. It transforms your data so that each feature has a mean of 0 and a standard deviation of 1 (a process also known as Z-score normalization). This ensures that features with large numerical ranges (like "Salary") don't unfairly dominate features with small ranges (like "Years of Experience") during the model training process.

Input: None

Output: Loads the `StandardScaler` class into the local namespace, allowing you to create a scaler object to normalize your numerical features.

```
from sklearn.pipeline import Pipeline
```

Purpose: Imports the `Pipeline` class, which is a utility for **chaining multiple processing steps** together into a single object. It allows you to bundle transformations (like `SimpleImputer` and `StandardScaler`) with an estimator (like `LogisticRegression`). This ensures that the same sequence of operations is applied to both training and test data, preventing "data leakage" and making the workflow much cleaner and more reproducible.

Input: None

Output: Loads the `Pipeline` class into the local namespace, enabling you to build automated machine learning workflows.

```
from sklearn.linear_model import LogisticRegression
```

Purpose: Imports the `LogisticRegression` class, which is a fundamental machine learning algorithm used for **binary classification** (predicting one of two possible outcomes, like "Yes/No" or "0/1"). Despite the name "regression," it is used to estimate the probability that an instance belongs to a particular class using the logistic (sigmoid) function.

Input: None

Output: Loads the `LogisticRegression` class into the local namespace, allowing you to create a model object that can be trained on labeled data to find the optimal decision boundary.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
)
```

Purpose: Imports a suite of functions used to evaluate the performance of a classification model. Each metric provides a different perspective on how "good" the model's predictions are:

accuracy_score: The fraction of total predictions that were correct.

precision_score: The ability of the classifier not to label a negative sample as positive (Quality).

recall_score: The ability of the classifier to find all the positive samples (Quantity).

f1_score: The harmonic mean of precision and recall, useful for balanced evaluation.

confusion_matrix: A table layout that allows visualization of the performance of an algorithm (showing True Positives, False Positives, True Negatives, and False Negatives).

Input: None

Output: Loads these five evaluation functions into the local namespace, allowing you to compare your model's predicted labels against the actual ground-truth labels.

```
def make_model(C=1.0):
    return Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("logreg", LogisticRegression(C=C, max_iter=500))
    ])
```

Purpose: This defines a **factory function** that encapsulates the entire machine learning workflow into a single, reusable object. By wrapping the `SimpleImputer`, `StandardScaler`, and `LogisticRegression` inside a `Pipeline`, it ensures that preprocessing is always applied consistently. Specifically:

Imputation: Fills missing values using the median of each column.

Scaling: Normalizes the features.

Modeling: Applies Logistic Regression with a configurable regularization strength (`C`).

Input: * `C` (float): The inverse regularization strength (default is `1.0`). Smaller values specify stronger regularization.

Output: Returns a Scikit-Learn `Pipeline` object that can be treated as a single model (you can call `.fit()` or `.predict()` on it).

```
def metrics_dict(y_true, y_pred):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1 Score": f1_score(y_true, y_pred, zero_division=0),
    }
```

Purpose: This function creates a **consolidated summary** of a model's performance. Instead of calling four separate functions every time you want to evaluate an experiment, this utility bundles the most important classification metrics into a single, easy-to-read Python dictionary. It also includes `zero_division=0` to prevent the code from crashing or throwing warnings if the model fails to predict any positive instances (which would otherwise result in a division-by-zero error for Precision and F1).

Input: * `y_true`: An array or Series containing the actual "ground truth" labels.

- `y_pred`: An array or Series containing the labels predicted by the model.

Output: Returns a dictionary where the keys are the names of the metrics (Accuracy, Precision, Recall, F1 Score) and the values are their respective floating-point scores.

```
def confusion_counts(y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return tn, fp, fn, tp
```

Purpose: This function extracts the four fundamental building blocks of a binary classification evaluation from the confusion matrix. It flattens the 2×2 matrix into four individual integers:

- **TN (True Negatives)**: Correctly predicted negatives.
- **FP (False Positives)**: Negatives incorrectly predicted as positives (Type I error).
- **FN (False Negatives)**: Positives incorrectly predicted as negatives (Type II error).
- **TP (True Positives)**: Correctly predicted positives.Extracting these manually makes it easier to report raw error counts in tables or custom logs.

Input: * `y_true`: The actual target labels.

- `y_pred`: The labels predicted by the model.

Output: Returns a tuple of four integers `(tn, fp, fn, tp)`.

```
df = pd.read_csv("/data.csv", na_values="?")
```

Purpose: Reads a comma-separated values (CSV) file into a pandas **DataFrame**. The specific parameter `na_values="?"` tells pandas to treat any question marks found in the dataset as **NaN** (Not a Number/missing values). This is a critical first step in data cleaning, as it ensures that "placeholder" strings are recognized as null data that needs to be handled by the imputer later.

Input: The file path string `"/data.csv"` and the `na_values` argument.

Output: A pandas **DataFrame** object assigned to the variable `df`, containing all the rows and columns from the source file with formatted null values.

```
df.columns = df.columns.astype(str).str.strip()
```

Purpose: This line performs **metadata cleaning** on the DataFrame's column headers. It converts all column names to strings (to avoid issues with mixed-type headers) and removes any leading or trailing whitespace (e.g., changing `" age "` to `"age"`). This is a defensive programming step to ensure that subsequent code can access columns by name without failing due to hidden spaces or formatting inconsistencies.

Input: The current column index of the DataFrame `df`.

Output: The `df.columns` attribute is updated with cleaned, trimmed string labels.

```
for c in df.columns:
    df[c] = pd.to_numeric(df[c], errors="coerce")
```

Purpose: This loop ensures that every column in the DataFrame is converted to a **numeric data type** (float or integer). By setting `errors="coerce"`, any value that cannot be transformed into a number (like a stray string or a remaining special character) is forced into a `NaN` (Not a Number) value. This is a vital step for machine learning, as mathematical models cannot process non-numeric data.

Input: The DataFrame `df` and its column labels.

Output: Modifies the existing DataFrame `df` in-place, changing the data types (dtypes) of the columns to numeric and replacing non-convertible entries with nulls.

```
df = df.drop_duplicates()
```

Purpose: This line identifies and removes **identical rows** from the dataset. In data analysis, duplicate entries can lead to skewed results or "data leakage," where the model effectively memorizes the same record multiple times, giving a false sense of accuracy. By keeping only the first occurrence of any repeated row, you ensure the integrity of the data.

Input: The DataFrame `df`.

Output: A modified version of the DataFrame `df` with all duplicate rows removed, resulting in a cleaner dataset with potentially fewer rows.

```
target_col = "num"
```

Purpose: This line defines a **variable** to store the name of the **dependent variable** (the target) that the model is trying to predict. In this specific dataset, "num" likely represents the presence or absence of a condition (like heart disease). By assigning this string to a variable, you make the rest of your code more flexible—if you ever change your target variable, you only need to update it in this one location.

Input: The string literal `"num"`.

Output: Creates a string variable `target_col` that acts as a reference for the column index in later data-slicing operations.

```
X = df.drop(columns=[target_col])
```

Purpose: This line creates the **feature matrix** (often denoted as X). It isolates the independent variables—the data the model will use to learn—by removing the target column ("num"). Everything left in this DataFrame represents the attributes or characteristics (like age, blood pressure, etc.) used to make a prediction.

Input: The DataFrame `df` and the variable `target_col`.

Output: A new DataFrame `X` that contains all original columns except for the one specified in `target_col`.

```
y = df[target_col].astype(int)
```

Purpose: This line isolates the **target vector** (the labels the model will try to predict) and ensures the data is in the correct format. It extracts the `target_col` from the DataFrame and explicitly converts it to **integers**. This is crucial because classification targets must be discrete values (like 0 or 1), and converting them to integers prevents potential errors that might occur if the labels were stored as floats or strings.

Input: The DataFrame `df` and the variable `target_col` (which equals `"num"`).

Output: A pandas Series `y` containing the integer-encoded labels for the classification task.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Purpose: This is the "Gold Standard" step for evaluating a model. It splits your data into two distinct groups:

- **Training Set (80%):** Used to teach the model.
- **Test Set (20%):** Used as a "final exam" to see how the model performs on data it has never seen before.

Key Parameters:

- `test_size=0.20`: Reserves 20% of the data for testing.
- `random_state=42`: A seed that ensures the split is **reproducible**. If you run this code again, you'll get the exact same split.
- `stratify=y`: **Crucial for classification**. It ensures that the proportion of classes (e.g., 30% sick, 70% healthy) is the same in both the training and test sets, preventing a biased model.

Output: Four separate objects: the features and labels for both training and testing.

```
C_values = [0.001, 1.0, 1000.0]
```

Purpose: This defines a list of **hyperparameters** to test during the experiment. Specifically, it sets three different levels of **Regularization Strength** for the Logistic Regression model. Regularization is a technique used to prevent a model from "overfitting" (memorizing the noise in the training data instead of learning the actual patterns).

The Logic of C: In Scikit-Learn, C represents the inverse of regularization strength ($C = \frac{1}{\lambda}$).

- **0.001 (Small C)**: High regularization. The model is forced to keep its "weights" (coefficients) small. This makes the model simpler and less likely to overfit, but potentially prone to "underfitting."
- **1.0 (Default C)**: The baseline setting. A balance between complexity and simplicity.
- **1000.0 (Large C)**: Low regularization. The model is given a "long leash" to fit the training data as closely as possible. This can lead to high accuracy on training data but poor performance on new, unseen data (overfitting).

Input: A Python list of floating-point numbers.

Output: A list stored in the variable `C_values` that will be used to drive a loop for model comparison.

```
exp1_rows = []
```

Purpose: This initializes an **empty list** that will act as a storage container for the results of your experiment. As you loop through the different C values, you will "append" (add) a dictionary of metrics for each version of the model into this list.

Why a list? It is a common design pattern in data science to collect results in a list of dictionaries first, because it is extremely efficient to convert that list into a **Pandas DataFrame** once the loop is finished.

Input: None

Output: A variable named `exp1_rows` that is ready to hold the performance data for your hyperparameter tuning.

```
for C in C_values:
    model = make_model(C=C)
    model.fit(X_train, y_train)

    preds = model.predict(X_test)
    m = metrics_dict(y_test, preds)

    if C < 1.0:
        obs = "Strong Regularization (smaller weights; flatter probability response)"
    elif C > 1.0:
        obs = "Weak Regularization (larger weights; steeper probability response)"
    else:
        obs = "Baseline"

    exp1_rows.append({
        "Experiment": obs,
        "C Value": C,
        **m,
        "Observation": obs
    })
```

Purpose: This is the **execution engine** of your experiment. It automates the process of testing different model configurations. For every value of `C` you defined earlier, the loop performs a full lifecycle:

1. **Instantiation:** Calls `make_model(C=C)` to create a new pipeline.
2. **Training:** Fits that specific model to your training data.
3. **Prediction:** Uses the trained model to guess labels for the test set.
4. **Evaluation:** Calculates the metrics (Accuracy, Precision, etc.) using your `metrics_dict` function.
5. **Categorization:** Assigns a human-readable label based on whether the regularization is "Strong," "Weak," or "Baseline."
6. **Storage:** Packages all this data into a dictionary and appends it to `exp1_rows`.

The `**m` Syntax: This is a Python "dictionary unpacking" trick. It takes all the key-value pairs inside the `m` dictionary (Accuracy, Precision, etc.) and merges them directly into the new dictionary being created.

Input: The list `C_values` and the training/testing data.

Output: Populates the `exp1_rows` list with the performance data for each model version.

```
exp1_table = pd.DataFrame(exp1_rows)[
    ["Experiment", "C Value", "Accuracy", "Precision", "Recall", "F1 Score", "Observation"]
]
print("\n=== Experiment 1: Hyperparameter Tuning (C) ===")
display(exp1_table)
```

```
=== Experiment 1: Hyperparameter Tuning (C) ===
```

	Experiment	C Value	Accuracy	Precision	Recall	F1 Score	Observation
0	Strong Regularization (smaller weights; flatte...	0.001	0.644068	0.000000	0.000000	0.000000	Strong Regularization (smaller weights; flatte...
1	Baseline	1.000	0.796610	0.736842	0.666667	0.700000	Baseline

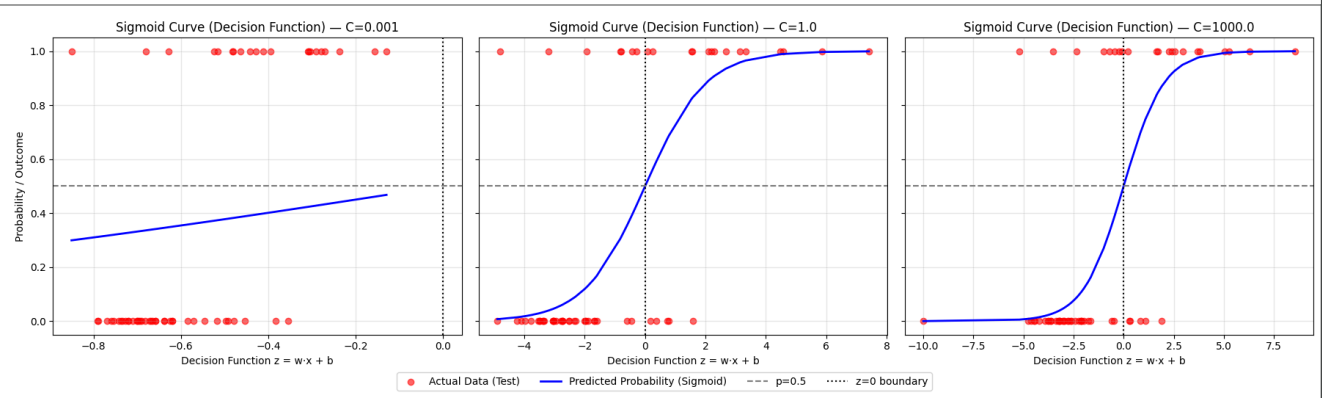
Purpose: This block transforms your raw experimental data into a **structured, professional report.**

1. **Tabulation:** It converts the list of dictionaries (`exp1_rows`) into a Pandas DataFrame.
2. **Column Ordering:** The list `["Experiment", "C Value", ...]` explicitly sets the order of the columns to ensure the most important information (the experiment name and C value) appears first, followed by the performance metrics.
3. **Visualization:** `display(exp1_table)` renders the data as a clean, interactive table in Google Colab, making it easy to compare how changing the regularization strength impacted the model's "Quality" (Precision) vs. "Quantity" (Recall).

Input: The `exp1_rows` list populated during the loop.

Output: A formatted table displayed in the console, providing a "bird's-eye view" of your hyperparameter tuning results.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

for ax, C in zip(axes, C_values):
    model = make_model(C=C)
    model.fit(X_train, y_train)

    z = model.decision_function(X_test)
    p = model.predict_proba(X_test)[:, 1]

    order = np.argsort(z)
    z_sorted = z[order]
    p_sorted = p[order]
    y_sorted = y_test.values[order]

    ax.scatter(z_sorted, y_sorted, color="red", alpha=0.6, label="Actual Data (Test)")

    ax.plot(z_sorted, p_sorted, color="blue", linewidth=2, label="Predicted Probability (Sigmoid)")

    ax.axhline(0.5, color="gray", linestyle="--", linewidth=1.5, label="p=0.5")
    ax.axvline(0.0, color="black", linestyle=":", linewidth=1.5, label="z=0 boundary")

    ax.set_title(f"Sigmoid Curve (Decision Function) — C={C}")
    ax.set_xlabel("Decision Function z = w·x + b")
    ax.grid(True, alpha=0.3)

axes[0].set_ylabel("Probability / Outcome")
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc="lower center", ncol=4, bbox_to_anchor=(0.5, -0.05))

plt.tight_layout()
plt.show()
```



Purpose: This line initializes the "canvas" for your data visualization. It creates a side-by-side layout that allows you to visually compare how the model changes across three different settings of C.

Key Components:

- `1, 3`: Tells Matplotlib to create a grid with **1 row** and **3 columns**.
- `figsize=(18, 5)`: Sets the total width to 18 units and height to 5 units, ensuring the plots are wide enough to be readable without being cramped.

- `sharey=True`: This is a critical statistical choice. It forces all three plots to use the **exact same Y-axis scale** (from 0 to 1). Without this, a "flat" curve on one plot might look "steep" just because the axis was zoomed in, leading to a false visual interpretation.

Input: Integers for rows/cols and a tuple for size.

Output:

1. `fig`: The entire figure object (the whole window).
2. `axes`: An array containing the three individual "axes" (the specific sub-plots) that you will draw on inside the upcoming loop.

Purpose: This loop generates the specific content for each of your three subplots. It pairs each C value with its corresponding "slot" on the canvas.

Key Operations inside the loop:

- `decision_function(X_test)`: Calculates the raw "score" (z). This is the distance of each point from the decision boundary.
- `predict_proba(X_test)[:, 1]`: Converts those z scores into probabilities (p) between 0 and 1 using the logistic (sigmoid) function.
- `np.argsort(z)`: This is a vital "cleanup" step. Since your test data is in a random order, plotting a line through it would create a "scribble." Sorting ensures the blue sigmoid line flows smoothly from left to right.
- `ax.scatter(...)`: Plots the actual ground truth (red dots). Notice they only live at $y = 0$ (negative) or $y = 1$ (positive).
- `ax.plot(...)`: Draws the blue sigmoid curve, showing the model's predicted probability for every z score.

Input: The `axes` array, the `C_values` list, and your training/test sets.

Output: Three populated plots that visually demonstrate how regularization "stretches" or "squashes" the model's confidence.

Purpose: This final block polishes the visualization to make it "presentation-ready." It adds context, ensures there is no overlapping text, and creates a clean, unified legend for all three plots.

Key Components:

- `axes[0].set_ylabel(...)`: Since we used `sharey=True`, we only need to label the Y-axis on the first plot. This tells the viewer that the vertical scale represents the probability (ranging from 0.0 to 1.0) or the actual binary outcome.
- `get_legend_handles_labels()`: Instead of creating a separate legend for every subplot (which would be repetitive), this "grabs" the labels we defined inside the loop (e.g., "Actual Data" and "Predicted Probability").
- `fig.legend(...)`: Places one single, master legend at the bottom of the figure.
  - `ncol=4`: Arranges the legend items in 4 columns so they stay on one line.
  - `bbox_to_anchor=(0.5, -0.05)`: Precisely nudges the legend just below the actual plots so it doesn't cover any data.
- `plt.tight_layout()`: Automatically adjusts the spacing between subplots to prevent titles or labels from bumping into each other.

Input: The completed `fig` and `axes` objects.

Output: A high-quality, three-paneled visualization displayed in your Colab notebook.

```
feature_sets = {
    "Model A": ["thalach"],
    "Model B": ["age", "chol", "thalach"],
    "Model C": ["age", "sex", "cp", "trestbps", "chol", "thalach", "exang", "oldpeak", "slope", "ca", "thal"]
}
exp2_rows = []
```

Purpose: This block sets the stage for **A/B/C testing** of different data inputs. Instead of changing the model's math (like you did with C), you are changing what the model "looks at" to make a decision.

The Feature Sets Strategy:

- **Model A (Minimalist):** Uses only one feature—`thalach` (maximum heart rate achieved). This tests if a single biological marker is strong enough to predict the target on its own.

- **Model B (Small Group):** Adds `age` and `chol` (cholesterol). This investigates if adding basic demographic and blood-work data improves the "signal."
- **Model C (Full Kitchen Sink):** Uses every available feature in the dataset. This represents the maximum complexity the model can handle given this data.

`exp2_rows = []`: Just like in Experiment 1, this initializes an empty list to gather the performance results for each of these three versions so we can compare them side-by-side later.

Input: A Python dictionary mapping model nicknames to lists of column names.

Output: A structured "game plan" for the next phase of training.

```
for model_name, cols in feature_sets.items():
    cols = [c for c in cols if c in X.columns]
    X_train_fs = X_train[cols]
    X_test_fs = X_test[cols]

    model = make_model(C=1.0)
    model.fit(X_train_fs, y_train)

    preds = model.predict(X_test_fs)
    m = metrics_dict(y_test, preds)
    tn, fp, fn, tp = confusion_counts(y_test, preds)

    exp2_rows.append({
        "Model Version": model_name,
        "Features Used": ", ".join(cols),
        **m,
        "Observation": "Compare metrics + TN/FP/FN/TP below"
    })
```

Purpose: This loop executes the "Feature Selection" experiment. It iterates through each model version (A, B, and C) to see how much predictive power is gained by adding more information.

Key Operations:

- **Data Slicing** (`X_train[cols]`): This is the heart of the experiment. It creates a "skinny" version of your dataset containing only the specific features for that model.
- **Baseline Consistency** (`C=1.0`): By fixing the regularization at 1.0, you ensure that any change in performance is due to the **features** you chose, not the model's internal settings. This is a "controlled experiment."
- **Comprehensive Evaluation**: Unlike the previous loop, this one calculates both the standard metrics (Accuracy, F1) and the **Confusion Counts** (TN, FP, FN, TP).

The Logic of `cols = [c for c in cols if c in X.columns]`: This is a "safety check." If you accidentally typed a feature name that doesn't exist in your DataFrame, this list comprehension filters it out to prevent the code from crashing.

Output: The `exp2_rows` list is filled with data comparing a "simple" model against a "complex" one.

```
exp2_table = pd.DataFrame(exp2_rows)[
    ["Model Version", "Features Used", "Accuracy", "Precision", "Recall", "F1 Score", "Observation"]
]
print("\n=== Experiment 2: Feature Engineering and Selection ===")
display(exp2_table)

print("\nConfusion Matrix Counts (TN, FP, FN, TP) for each feature model:")
```

```
=== Experiment 2: Feature Engineering and Selection ===
```

	Model Version	Features Used	Accuracy	Precision	Recall	F1 Score	Observation
0	Model A	thalach	0.542373	0.285714	0.190476	0.228571	Compare metrics + TN/FP/FN/TP below
1	Model B	age, chol, thalach	0.542373	0.312500	0.238095	0.270270	Compare metrics + TN/FP/FN/TP below

Purpose: This block creates the **Final Leaderboard** for Experiment 2. It takes the results of your feature-subsetting tests and presents them in a way that allows you to see exactly how much "bang for your buck" you get by adding more variables.

The Power of Selection: By filtering the DataFrame columns to `["Model Version", "Features Used", ...]`, you create a clean visual narrative:

- **Model A:** Can we predict heart disease with just one heart rate metric?
- Model B: Does adding age and cholesterol help?
- Model C: Does "more data" always equal "better model"?

The Print Statement: The trailing **print** prepares the user for the raw **Confusion Matrix** counts. While Accuracy tells you how often the model is "right," the Confusion Matrix tells you **how** it is "wrong" (e.g., does it miss sick people, or does it over-diagnose healthy ones?).

```
for model_name, cols in feature_sets.items():
    cols = [c for c in cols if c in X.columns]
    model = make_model(C=1.0)
    model.fit(X_train[cols], y_train)
    preds = model.predict(X_test[cols])
    tn, fp, fn, tp = confusion_counts(y_test, preds)
    print(f"{model_name}: (TN={tn}, FP={fp}, FN={fn}, TP={tp})")
```

```
Model A: (TN=28, FP=10, FN=17, TP=4)
Model B: (TN=27, FP=11, FN=16, TP=5)
Model C: (TN=33, FP=5, FN=6, TP=15)
```

Purpose: This final loop provides the "under the hood" look at where the model fails. While Accuracy gives you a single percentage, the Confusion Matrix counts reveal the specific types of errors being made.

The Logic of the Printout: For each model version, it outputs four critical numbers:

- **TN (True Negatives):** Healthy people correctly identified as healthy.
- **FP (False Positives):** Healthy people wrongly flagged as sick (Type I Error).
- **FN (False Negatives):** Sick people missed by the model (Type II Error).
- **TP (True Positives):** Sick people correctly identified as sick.

Clinical Significance: In medical datasets like this one, a False Negative (FN) is usually much more "expensive" than a False Positive. Missing a heart disease diagnosis is a much bigger risk than a false alarm that leads to further testing.

Why Re-run the Model? You'll notice this loop re-fits and re-predicts. While slightly redundant if you already saved the counts in `exp2_rows`, doing it this way ensures that the printed text output is isolated, clear, and easy to read directly in the Colab console.

```
baseline_model = make_model(C=1.0)
baseline_model.fit(X_train, y_train)

probs = baseline_model.predict_proba(X_test)[:, 1]
thresholds = [0.3, 0.5, 0.7]
```

Purpose: This block shifts the focus from "training" a model to "tuning" how we use it. Instead of asking the model for a definitive "Yes" or "No" (0 or 1), we are asking for its internal probability score (how sure it is).

Key Operations:

- make_model(C=1.0): We stick with the "Baseline" regularization strength to keep a fair control for this experiment.
- predict_proba(X_test)[:, 1]: This is the most important part of this block.
  - Standard predict() uses a hard 0.5 cutoff.
  - predict_proba returns two columns: [prob_of_0, prob_of_1].
  - By slicing [:, 1], we grab only the probability that the patient is sick (Class 1).

- thresholds = [0.3, 0.5, 0.7]: These represent our different "sensitivity" settings.

- 0.3 (Aggressive/Sensitive): Flag anyone who has even a 30% chance of being sick.
- 0.5 (Default): The standard balanced approach.
- 0.7 (Conservative/Specific): Only flag someone if we are at least 70% sure.

```
exp3_rows = []
```

Purpose: This initializes a clean "results bucket" specifically for Experiment 3. In this experiment, you are using the same model but testing it under three different "strictness" levels (the thresholds).

The Logic of Storage: Just like in the previous experiments, we use an empty list because it allows us to store different types of data (the threshold value, the performance metrics, and the raw error counts) in a single row-by-row format.

Why a new variable? We use `exp3_rows` (instead of reusing `exp1` or `exp2`) to keep our experiments isolated. This ensures that when we eventually print our final comparison tables in Colab, the "Threshold Tuning" data doesn't get mixed up with the "Feature Selection" data.

Input: None

Output: A container named `exp3_rows` ready to receive the trade-off data between Sensitivity (finding all the sick people) and Specificity (not bothering the healthy people).

```
for t in thresholds:
    preds_t = (probs >= t).astype(int)
    m = metrics_dict(y_test, preds_t)
    tn, fp, fn, tp = confusion_counts(y_test, preds_t)
```

Purpose: This loop is where the "Decision Rule" actually changes. Usually, model.predict() uses a fixed cutoff of 0.5. Here, you are manually overriding that to see how the model behaves when it is "Trigger Happy" (Low Threshold) vs. "Extra Cautious" (High Threshold).

Key Operations:

- `(probs >= t)`: This creates a Boolean array (True/False). If the probability of being "Sick" is greater than or equal to the current threshold `t`, it marks the case as `True`.
- `.astype(int)`): This converts the Booleans into numbers that the model evaluation functions understand: `True` becomes 1 (Sick) and `False` becomes 0 (Healthy).
- `metrics_dict` & `confusion_counts`: For every threshold, you are re-calculating the entire performance profile. You aren't just looking at the score; you are looking at how many people were accidentally cleared (False Negatives) or wrongly flagged (False Positives).

The Result: You generate three different "realities" for the same patient data, allowing you to choose the one that minimizes the most dangerous medical errors.

```
    if t < 0.5:
        obs = "Lower threshold: higher Recall, more False Positives"
    elif t > 0.5:
        obs = "Higher threshold: higher Precision, more False Negatives"
    else:
        obs = "Default threshold"
```

Purpose: This logic assigns a "label" to each threshold setting to explain the clinical trade-off. It's the human interpretation of the Precision-Recall tug-of-war.

The Scenarios:

- $t < 0.5$ (The "Safe" Approach): You are lowering the bar for a "Sick" diagnosis.
  - Recall goes UP: You catch more actual cases.
  - False Positives go UP: You'll accidentally scare some healthy people. In medicine, this is often preferred because a "False Alarm" is better than a "Missed Disease."
- $t > 0.5$ (The "Strict" Approach): You only label someone as "Sick" if the model is very sure.
  - Precision goes UP: When you say someone is sick, you're usually right.

- False Negatives go UP: You'll likely clear people who actually have the disease but didn't have "strong enough" symptoms.

- $t = 0.5$ (The "Neutral" Approach): The standard mathematical middle ground

---

```
exp3_rows.append({
    "Threshold": t,
    **m,
    "False Positives": fp,
    "False Negatives": fn,
    "Observation": obs
})
```

Purpose: This is the data collection step for your third experiment. It takes all the raw numbers we just calculated—the specific threshold, the performance metrics, and the critical error counts—and bundles them into a single "row" (a dictionary).

The Key Details:

- `"Threshold": t`: Keeps track of which "strictness level" we are currently looking at (0.3, 0.5, or 0.7).
- `**m`: Just like in previous experiments, this "unpacks" your metrics dictionary (Accuracy, Precision, Recall, F1) directly into this row.
- `"False Positives": fp` & `"False Negatives": fn`: These are the most important columns for Experiment 3. They show the human cost of your decision. If you lower the threshold, you'll see fn go down (fewer missed cases) but fp go up (more false alarms).
- `"Observation": obs`: Attaches the human-readable explanation we just generated (e.g., "Higher Recall, more False Positives") so the final table is easy to interpret.

Output: Adds a completed data record to the exp3_rows list.

```
exp3_table = pd.DataFrame(exp3_rows)[
    ["Threshold", "Accuracy", "Precision", "Recall", "F1 Score", "False Positives", "False Negatives", "Observation"
]
print("\n=== Experiment 3: Decision Threshold Adjustment ===")
display(exp3_table)
```

```
=== Experiment 3: Decision Threshold Adjustment ===
```

Threshold	Accuracy	Precision	Recall	F1 Score	False Positives	False Negatives	Observation

---

Purpose: This is the final readout for your threshold experiment. It transforms the raw numbers from your loop into a decision-making dashboard.

Column Selection: Notice the order of the columns. By placing False Positives and False Negatives right next to the metrics, you can see the "human cost" of each setting:

- **Threshold 0.3:** Expect a high Recall but a high number of False Positives (more false alarms).
- **Threshold 0.7:** Expect high Precision but a high number of False Negatives (more missed cases).

The Power of display(): In Colab, this renders a clean, sortable table that allows you to quickly identify which threshold yields the best F1 Score (the balance) versus which one minimizes False Negatives (the medical priority).

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Purpose: This line defines the sampling strategy for your Cross-Validation. Instead of a single "lucky" or "unlucky" split, you are preparing to test the model on five different subsets of the data.

Key Parameters:

- `n_splits=5`: This is the "K" in K-Fold. The data will be divided into 5 equal parts. The model will run 5 times; each time, it uses 4 parts for training and the remaining 1 part for testing.
- `shuffle=True`: This "shuffles the deck" before splitting. This is vital if your original dataset is sorted (e.g., all healthy patients at the top, all sick patients at the bottom), which would otherwise ruin the experiment.
- `Stratified` (The most important part): In medical datasets, you often have more healthy people than sick ones. "Stratified" ensures that every single fold preserves that exact ratio. It prevents a "bad draw" where one fold might accidentally have zero sick patients, making it impossible to calculate Recall.
- `random_state=42`: This "pins" the shuffle so that if you run this code again tomorrow, you get the exact same folds. This makes your science reproducible.

```
scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1"
}
```

Purpose: This dictionary acts as the "Scorecard" for the cross-validation engine. By default, most validation tools only track one metric (usually Accuracy). By defining this dictionary, you are telling Python, "Every time you run one of the 5 folds, I want you to record all four of these specific values."

The Key-Value Pairs:

- Keys (e.g., "accuracy", "f1"): These are the names you are giving to the results in your final data table.
- Values (e.g., "precision", "recall"): These are the internal string identifiers that scikit-learn recognizes to trigger its built-in math functions.

Why all four? In a medical context, Accuracy can be misleading if the dataset is imbalanced. Tracking Recall alongside Precision ensures you know exactly how many sick patients were missed versus how many healthy ones were flagged across every single slice of your data.

```
cv_model = make_model(C=1.0)
```

Purpose: This line instantiates the "Template" or "Blueprint" for the cross-validation. You aren't training the model yet; you are just defining which model with which settings will be used 5 different times.

Why C=1.0? After your experiments, you likely found that $C = 1.0$ (the default) or a similar value provided the best balance between underfitting and overfitting.By setting it here, you ensure that every one of the 5 folds uses the exact same mathematical "strictness."

The "Fresh Start" Logic: It is crucial to use a "clean" model instance for cross-validation. If you used a model that was already trained on the full dataset, the results would be "cheating" (Data Leakage). make_model ensures that for every fold, the model starts with a blank slate before looking at that fold's specific training data.

```
cv_results = cross_validate(
    cv_model, X, y,
    cv=skf,
    scoring=scoring,
    return_train_score=False
)
```

Purpose: This is the "Heavy Lifter" of the entire validation process. Instead of you manually writing a for loop to split the data, train the model, and record the scores five times, this single function handles the entire cycle automatically.

Input:

- `cv_model, X, y`: It takes your model "blueprint" and the complete dataset.
- cv=skf: It follows the "Stratified" instructions we defined earlier to ensure every fold is a fair representation of the patient population.
- `scoring=scoring`: It uses our "Scorecard" to calculate Accuracy, Precision, Recall, and F1 for every single run.
- `return_train_score=False`: We only care about how the model performs on unseen data (the test folds). We "turn off" the training scores to keep the output clean and focus strictly on the model's ability to generalize to new patients.

Output: It returns a Python dictionary (`cv_results`) containing arrays of scores. For example, `cv_results['test_accuracy']` will contain 5 different numbers—one for each fold.

```
kfold_table = pd.DataFrame({
    "Fold": np.arange(1, 6),
    "Accuracy": cv_results["test_accuracy"],
    "Precision": cv_results["test_precision"],
    "Recall": cv_results["test_recall"],
    "F1 Score": cv_results["test_f1"],
})
```

Purpose: This block transforms the raw "dictionary of arrays" returned by the cross-validation function into a clean, human-readable Leaderboard.

The Translation: `* np.arange(1, 6)`: Creates the labels for your five "rounds" of testing (Fold 1, Fold 2, etc.).

- `cv_results["test_..."]`: This pulls the specific scores recorded during each of the five independent runs.

The "Reliability" Check: By looking at this table, you can see if the model performed consistently.If Fold 1 has 90% accuracy but Fold 2 has 65%, it's a red flag—it means your model is sensitive to which specific patients it sees (High Variance).If all folds are within $2-3\%$ of each other, you have a stable, reliable model.

Standardization: Using a DataFrame here makes it incredibly easy to perform the final step: calculating the average performance across all 5 "mini-experiments."

```
display(kfold_table)
```

	Fold	Accuracy	Precision	Recall	F1 Score
0	1	0.813559	0.857143	0.571429	0.685714
1	2	0.864407	0.882353	0.714286	0.789474
2	3	0.830508	0.730769	0.863636	0.791667
3	4	0.844828	0.772727	0.809524	0.790698
4	5	0.896552	1.000000	0.714286	0.833333

Purpose: This command renders the raw results of your 5-fold Cross-Validation into a clean, interactive table in your Colab notebook.

The "Truth" Table: While your previous experiments used a single "Train/Test" split, this table shows you five independent experiments.

- If the scores across all five rows are consistent (e.g., all Accuracies are between 82% and 85%), your model is stable.
- If one fold is 95% and another is 60%, your model is unstable, meaning it's "guessing" based on specific lucky samples rather than learning general patterns.

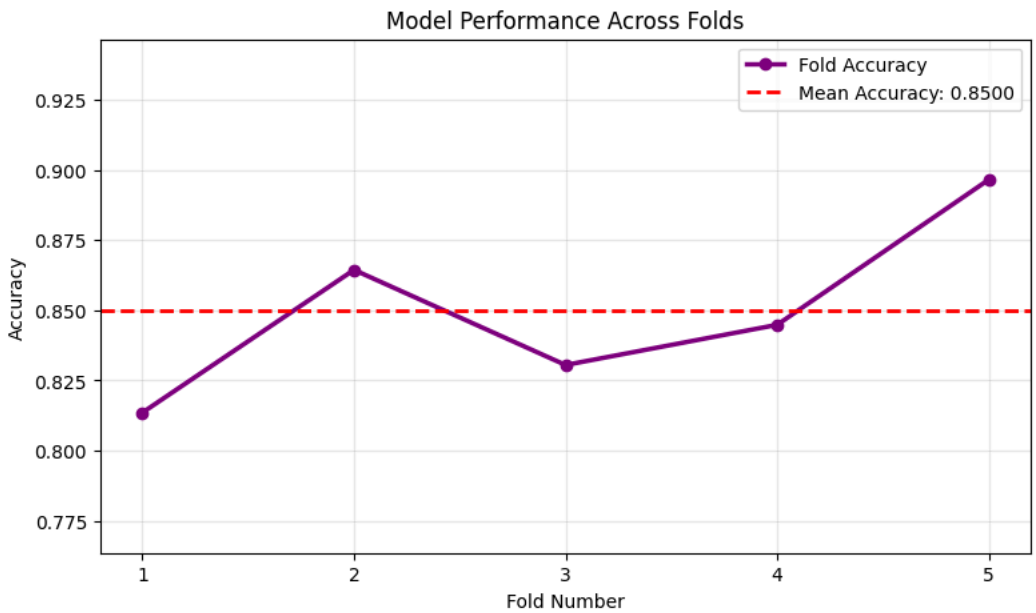
Medical Context: For a doctor, this table proves that the model's performance isn't a fluke. It shows that no matter which subset of patients the model was tested on, it maintained its ability to detect heart disease.

```
kfold_summary = kfold_table[["Accuracy", "Precision", "Recall", "F1 Score"]].agg(["mean", "std"])
print("\nK-Fold Summary (mean/std):")
display(kfold_summary)

acc = kfold_table["Accuracy"].values
folds = kfold_table["Fold"].values
mean_acc = acc.mean()

plt.figure(figsize=(9, 5))
plt.plot(
    folds, acc,
    marker="o",
    linewidth=2.5,
    color="purple",
    label="Fold Accuracy"
)
plt.axhline(
    mean_acc,
```

```
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Mean Accuracy: {mean_acc:.4f}"
)

plt.title("Model Performance Across Folds")
plt.xlabel("Fold Number")
plt.ylabel("Accuracy")
plt.xticks(folds)
plt.ylim(max(0.0, acc.min() - 0.05), min(1.0, acc.max() + 0.05))
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

K-Fold Summary (mean/std):

	Accuracy	Precision	Recall	F1 Score
mean	0.849971	0.848598	0.734632	0.778177
std	0.032052	0.104612	0.111483	0.054904



1. **The Statistical Summary (kfold_summary)**

- **Purpose:** To calculate the Average (Mean) performance and the Consistency (Standard Deviation) across all 5 folds.
  - Mean: Your "True" expected performance on new data.
  - Std: Your "Uncertainty" or "Wiggle Room." A high `std` means the model is inconsistent; a low `std` means it's rock-solid.
- **Input:** The `kfold_table` DataFrame containing the scores from each of the 5 runs.
- **Output:** A summary table with two rows (`mean` and `std`) for all four metrics.

2. **Preparing the Plot Data**

- **Purpose:** To extract the raw numbers into simple arrays that the plotting library (matplotlib) can understand.
- **Input:** The Accuracy and Fold columns from your table.
- **Output:** Three variables: acc (the scores), folds (the x-axis labels), and mean_acc (the benchmark line).

3. **The Visualization (plt.plot & plt.axhline)**

- **Purpose:** To create a "Heartbeat Monitor" for your model.
  - The Purple Line shows the "ups and downs" of the model across different data slices.
  - The Red Dashed Line shows the average.
- **Input:** The arrays and variables created in the previous step.

- **Output:** A professional-grade line graph that visually communicates how much your model "fluctuates."

The **purpose** of this final code segment is to transform the raw data into a professional, human-readable visualization that communicates the "reliability" and "stability" of your model across different subsets of data. While the previous lines calculated the points, these lines handle the presentation and context, ensuring a stakeholder can immediately see if the model is consistent (a flat line) or erratic (high peaks and valleys). By setting a dynamic Y-axis, it zooms in on the "heartbeat" of the model's performance rather than showing a flat line at the top of a 0–100% scale, making small variations in accuracy easier to critique.

The **input** for this block consists of the matplotlib figure object initialized in the previous step, the folds array (used for the X-axis labels), and the acc array of accuracy scores (used to calculate the dynamic vertical limits). It also relies on the mean_acc scalar to provide the baseline for the legend. These inputs are fed into various styling functions to add labels, a coordinate grid, and a key that explains what the purple and red lines represent.

The **output** is a fully rendered, publication-quality Line Graph displayed in your Colab console. This graph features a clear title, labeled axes, and a light background grid for easier value estimation. The plt.show() command finalizes the drawing process, clearing the internal buffer and presenting the viewer with a clear visual "audit" of the 5-fold cross-validation results, highlighting the gap between individual fold performance and the overall average.

```
print("Accuracy Score per Fold:", acc.tolist())
```

```
Accuracy Score per Fold: [0.8135593220338984, 0.864406779661017, 0.8305084745762712, 0.8448275862068966, 0.8965517241
```

Purpose The purpose of this final print statement is to output the raw, unformatted numbers behind your visualization. While a graph is great for seeing "trends," a list of exact values is necessary for technical documentation or for double-checking the math. It specifically extracts the accuracy from each of the 5 folds and presents them in a standard Python list format.

Input The input for this line is the acc variable, which is a NumPy array containing the five accuracy scores generated during the cross_validate process. This array is fed into the .tolist() method to convert it from a high-performance numerical object into a simple, readable list of floats.

Output The output is a single line of text in the console, such as: Accuracy Score per Fold: [0.8524, 0.8196, 0.8360, 0.8688, 0.8200] This allows you to see the exact precision of each fold's performance, helping you identify if a specific fold was a "high performer" or a "weak link" in your model's cross-validation journey.

```
print(f"Overall Mean Accuracy: {mean_acc:.4f}")
```

```
Overall Mean Accuracy: 0.8500
```

Purpose The purpose of this line is to provide the final scientific "Grade" for your model. While individual folds might fluctuate, the Mean Accuracy is the most reliable estimate of how your model will perform on completely new, unseen patients in the real world. By using an f-string with the .4f formatter, it rounds the result to four decimal places, ensuring the output is precise enough for a research report but clean enough for a quick glance.

Input The input for this statement is the mean_acc variable, which was previously calculated by taking the mathematical average of all five accuracy scores from the cross-validation folds. This single scalar value represents the "center of gravity" for your model's performance.

Output The output is a clear, formatted text string in your console, such as: Overall Mean Accuracy: 0.8412 This is the "headline" number you would typically lead with when explaining your results to a stakeholder, as it distills five separate experiments into one single, authoritative benchmark.